# 日历管不好系统实验报告

**系统名称：** 日历管不好系统
**开发者**：2025013092楼诗雨
**图形界面：** Tkinter
**第三方库：** lunarcalendar==0.0.9
**使用ai部分的声明**：为更好实现ui，在ui部分用了ai

## 目录


1. 系统架构
2. 基础功能实现
3. 创新功能实现
4. 数据结构与持久化
5. 鲁棒性


## 1. 系统架构

### 1.1 项目结构

```
日历系统/
├── main.py                    # 程序入口
├── config.py                  # 系统配置
├── requirements.txt           # 依赖库
├── models/                    # 数据模型层
│   ├── __init__.py
│   ├── event.py              # 事件类定义
│   └── date_utils.py         # 日期工具函数
├── storage/                   # 数据存储层
│   ├── __init__.py
│   └── event_storage.py      # 事件持久化
├── ui/                        # 用户界面层
│   ├── __init__.py
│   ├── main_window.py        # 主窗口控制器
│   ├── menu_view.py          # 主菜单视图
│   ├── calendar_view.py      # 月历打印视图
│   ├── date_query_view.py    # 日期查询视图
│   └── event_manage_view.py  # 事件管理视图
└── data/                      # 数据文件目录
    └── events.json           # 事件数据文件
```

### 1.2 整体架构设计

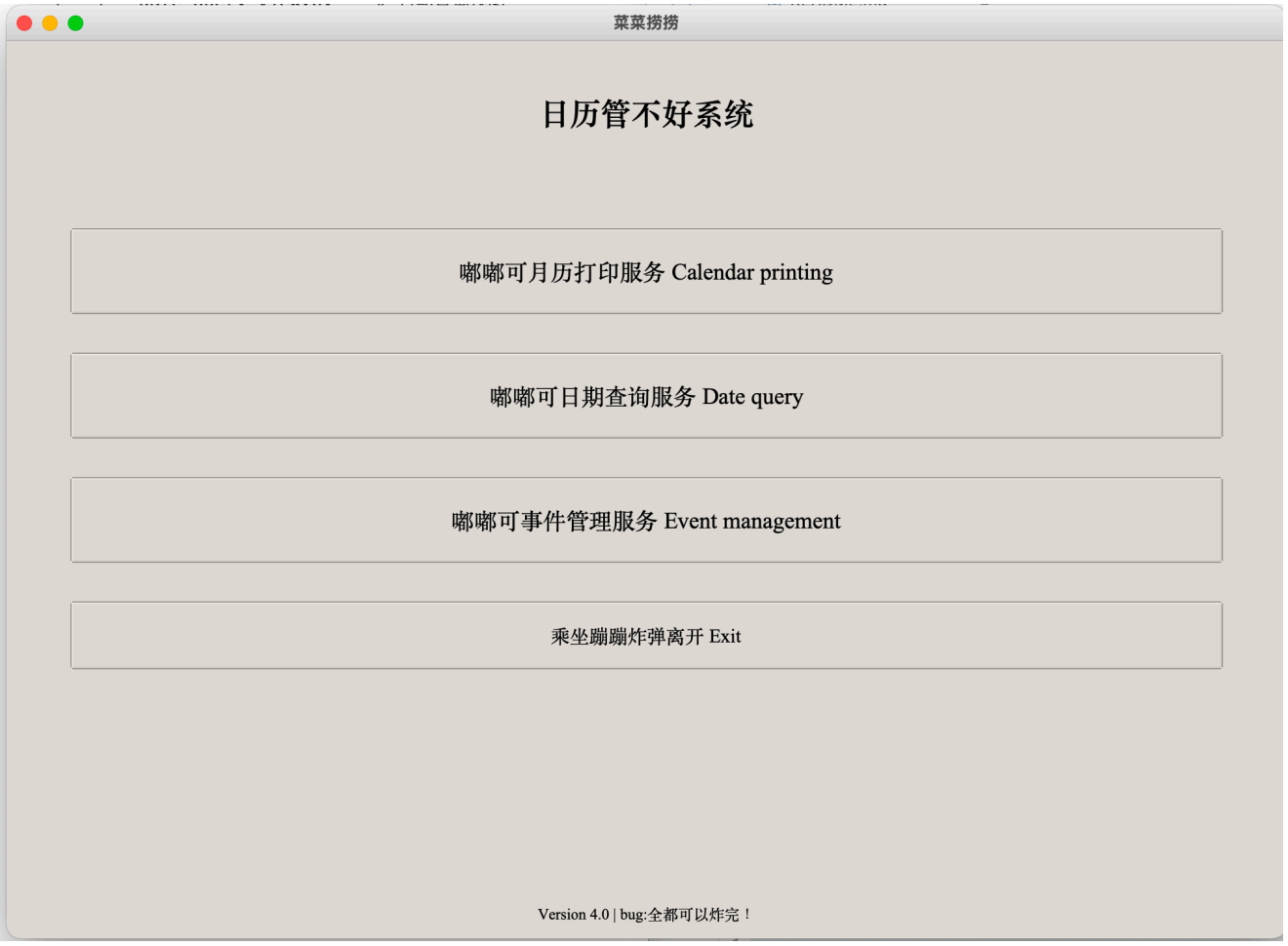
- **模型层：** `models/` 目录，包含事件数据模型和日期计算工具

- **视图层：** `ui/` 目录，包含用户界面组件们
- **控制器：** `ui/main_window.py` 中的 `MainWindow` 类，用来协调视图和模型

# 2. 基础功能实现

## 2.1 主菜单

ui实现：`ui/menu_view.py` 文件




**功能描述：**

（1）显示三个核心功能模块的按钮和一个退出按钮

（2）夹带了作者的许多私货，表达了作者的思乡之情

---

## 2.2月历打印

ui实现：`ui/calendar_view.py` 文件

主要用到的model：`models/date_utils.py` 文件

**功能描述：**
（1）显示指定年月的日历，深绿色和 ● 标记有事件的日期，浅绿色标记无事件的日期（初始条件默认显示当前月份的月历）
（2）点击某个日期可跳转到该日期查询界面
（3）自动统计本月事件的个数/某日事件的个数并显示

**核心方法：**

- 生成并显示月历
  ui层：`ui/calendar_view.py` 文件，`CalendarView` 类 `update_calendar()` 函数 #line132
  model层：`models/date_utils.py` 文件 ，`get_month_calendar()` 函数 #line112

**辅助方法：**
均位于model层：`models/date_utils` 文件

- is_run_year(year) 判断是否为闰年 #line7
- get_days_in_month(year, month) 获取指定年月的天数 #line11
- get_weekday(year, month, day) 计算星期几【使用蔡勒公式】 #line16

---

## 2.3 日期查询

### 2.3.1 在日期查询页面的日期信息

ui实现：`ui/date_query_view.py` 文件
主要用到的model： `models/date_utils.py` 文件

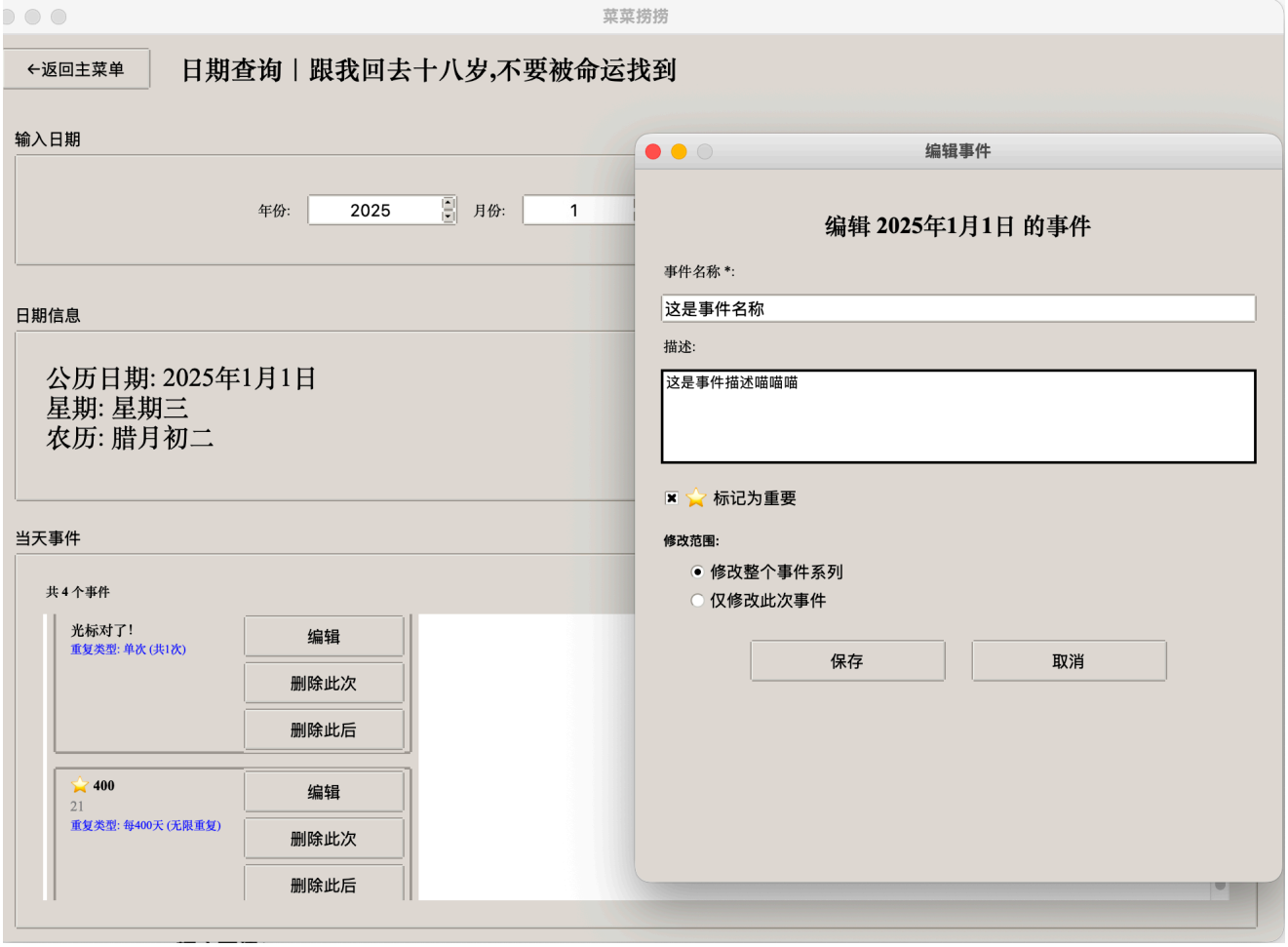
**功能描述：**
（1）查询某一天的星期几/农历日期（初始条件默认显示当前日期的查询结果）
(2) 列出当天事件，并提供编辑与删除此次或此后按钮

**核心方法：**

- 日期基础信息查询
  ui层： `ui/date_query_view.py` 文件， `DateQueryView` 类 `query_date()` 函数 #line127
  - 星期计算：使用蔡勒公式（见 2.2 节）
  - 农历转换：
    model层： `models/date_utils.py` 文件， `get_lunar_date()` 函数 #line37
  - 当日事件列表查询
    ui层： `ui/date_query_view.py` 文件， `DateQueryView` 类 `display_events()` 函数 #line163
    storage层： `storage/event_storage.py` 文件， `get_events_for_date()` 函数 #line88

### 2.3.2 在日期查询页面的事件编辑

ui实现：`ui/date_query_view.py` 文件
主要用到的model： `models/event.py` 文件




**功能描述：**
（1）可修改事件名称、描述、是否标记为重要
(2) 可选修改范围：整个事件系列或仅修改此次事件

**核心方法：**

- 仅修改此次事件时的分裂逻辑
  ui层： `ui/date_query_view.py` 文件， `DateQueryView` 类 `save_changes()` 函数 #line466
  model层： `models/event.py` 文件， `split_at_date()` 函数 #line127
  实现思路：

```
def split_at_date(self, split_year, split_month, split_day, new_id_func):
    """
    在指定日期分裂事件为三个部分的思路
    返回: (before_event, current_event, after_event)
    before_event: 指定日期之前的重复事件（如果有）
    current_event: 指定日期当天的单次事件
    after_event: 指定日期之后的重复事件（如果有）
    """
```

```
    # 单次事件不分裂
    # 创建之前的事件系列（设置结束日期）
    # 排除当天
    # 创建当天的单次事件
    # 创建之后的事件系列
    # 计算新的开始日期（下一次重复）
```

**辅助方法：**
均位于model层： `models/event.py` 文件

- _get_next_occurrence() 获取指定日期之后的下一次发生日期 #line193
- _adjust_repeat_count() 调整分裂后的重复次数 #line230

---

### 2.3.3 在日期查询页面的事件删除

ui实现：`ui/date_query_view.py` 文件
主要用到的model： `models/event.py` 文件
**功能描述：**
（1）可删除此次事件
(2) 可删除此后事件
**核心方法：**

- 仅修改此次事件时的分裂逻辑
  model层： `models/event.py` 文件， `delete_single_date()` 函数 #line285
  实现思路：

```
def delete_single_date(self, delete_year, delete_month, delete_day,
new_id_func):
    """
    删除单个日期的事件实例的思路
    返回: (before_event, after_event)
    """
    # 单次事件直接删除
    # 创建之前的事件（设置结束日期为删除日期）
    # 创建之后的事件
```

- 删除此后事件
  model层： `models/event.py` 文件， `delete_from_date()` 函数 #line255
  实现思路：

```
def delete_from_date(self, delete_year, delete_month, delete_day):
    """
    从指定日期开始删除的思路
    返回修改后的事件,如果完全删除返回None
```

```
    """
    # 单次事件直接删除
    # 设置结束日期为删除日期的前一次发生
    # 将删除日期加入排除列表
    # 检查是否还有有效的发生日期
    # 如果开始日期是重复事件的第一次，则完全删除
```

---

## 2.4 事件管理

ui实现：`ui/event_manage_view.py` 文件
主要用到的model：`models/event.py` 文件

**功能描述：**

（1）呈现所有事件，并提供事件总量统计
（2）增：添加事件
支持输入：事件名称/描述/开始日期/重复类型/重复次数/是否标🌟
（3）删：删除整个事件系列（右键）
（4）改：修改整个事件系列（双击或右键）
（5）查：通过关键词搜索相关事件

**核心方法：**

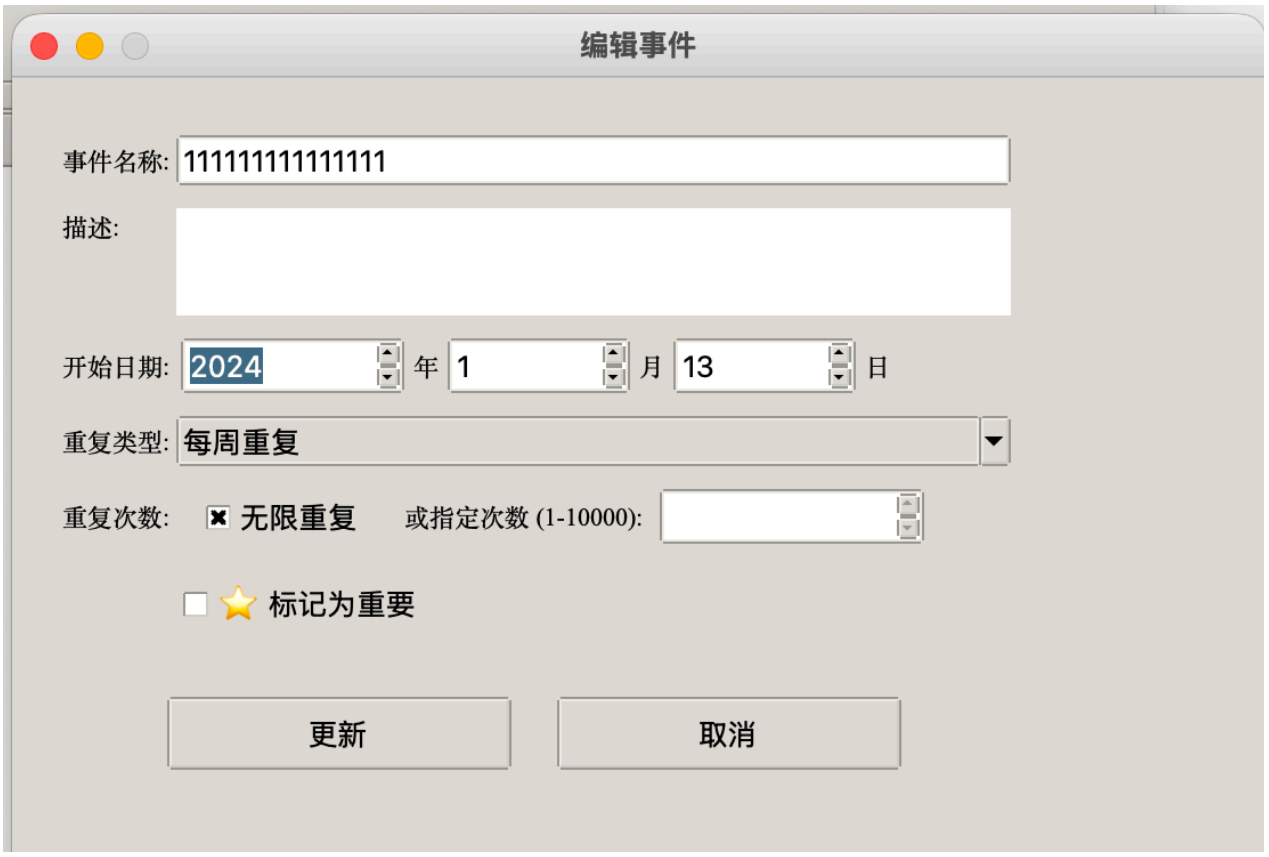
- 事件搜索
  ui层： `ui/event_manage_view.py` 文件， `EventManageView` 类
  model层： `models/event.py` 文件， `search_events()` 函数 #line94

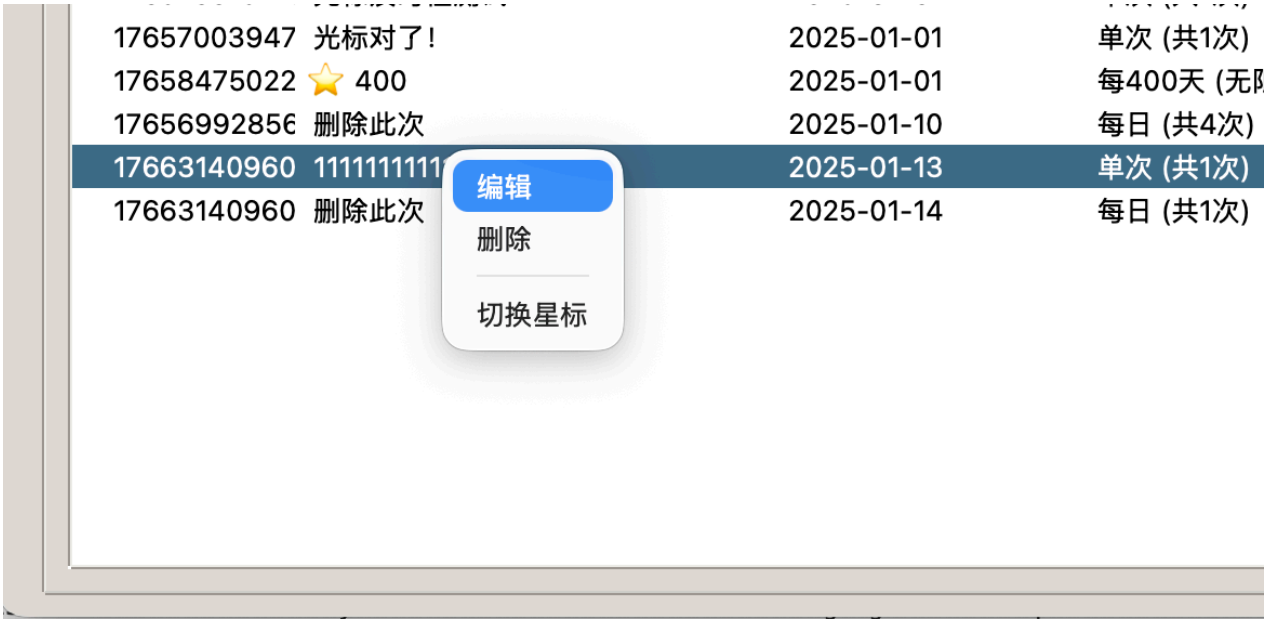
**辅助功能**：
实时搜索过滤：使用变量追踪机制，输入即时过滤

```
self.search_var.trace('w', lambda *args: self.refresh_event_list())
```

- 事件添加/修改
  **功能描述**:
  （1 右上角按钮添加
  （2）双击或右键选择修改
  通过 `EventDialog` 类 里mode='add'或'edit'实现
  ui层： `ui/event_manage_view.py` 文件， `EventDialog` 类
  model层： `models/event.py` 文件
  ui层 `EventDialog` 类 里的一些函数：
- on_repeat_change(self, event=None) 重复类型改变时的处理
- on_infinite_change(self) 无限重复复选框改变时的处理
- load_event_data(self) 加载事件数据（编辑模式）

- save_event(self) 保存事件




- 事件删除
  `ui/event_manage_view.py` 文件里的 `delete_selected_event()` 函数 #line196
- 事件切换星标
  `ui/event_manage_view.py` 文件里的 `toggle_star()` 函数 #line228

---

# 3. 创新功能实现

## 3.1 GUI设计

- 使用Tkinter作为GUI框架

- 采用clam主题
  **总体思路**

1. **Controller层**： `MainWindow` 类作为主控制器，负责：
   - 视图切换管理（通过 `show_view()` 方法）
   - 事件数据的统一管理
   - 视图间的数据传递
2. **View层**：多视图切换，使用 `place` 布局管理器实现视图叠加： `main_window.py`

```
for view in self.views.values():
    view.place(x=0, y=0, relwidth=1, relheight=1)
```

通过lift()和lower()方法控制视图显示顺序，实现无缝切换

**各个视图的简单概述**

- 主菜单视图 `menu_view.py`
  - 采用垂直布局，使用 `pack` 布局管理器
  - 四个主要功能按钮使用自定义样式（Large.TButton）
  - 使用lambda表达式实现按钮回调
- 月历打印视图 `calendar_view.py`
  - 使用7列网格表示一周
  - 使用 `grid` 布局管理器控制位置
- 日期查询视图 `date_query_view.py`
  - 日期输入使用Spinbox控件限制输入范围
  - 使用Canvas+Scrollbar实现可滚动区域
- 事件管理视图 `event_manage_view.py`
  - 使用Treeview数据表格

**统一的视图刷新机制** `main_window.py` 中

```
def apply_events_change(self, new_events):
    """统一的事件变更入口"""
    self.events = new_events
    self.storage.save_events(self.events)
    self.refresh_all_views()
```

所有数据修改操作都通过此方法保证内存、文件、视图三者的数据一致性， 防止视图数据不同步的问题

**部分输入框有自动全选**：焦点进入时全选文本，方便修改

```
entry.bind('<FocusIn>', lambda e: e.widget.selection_range(0, tk.END))
```

## 3.2 大跨度情况下的性能优化

| | | | | |
|---|---|---|---|---|
| 17656924775 | 【每月重复】月末测试 | 2025-01-31 | 每月 (共12次) | 31日在其他月份 |
| 17656925433 | 【每年重复】闰年测试 | 2023-02-28 | 每年 (无限) | 2.29 |
| 17656941958 ⭐ | 【大量重复测试】 | 2025-10-11 | 每52天 (共9999次) | 重复9999次 |
| 17656949382 ⭐ | 【长时间重复测试】 | 2025-10-11 | 每356000天 (共2次) | 致一千年后的我们 |

经测试可行

**代码位置：** `models/event.py` 文件，`occurs_on_date()` 方法 #line41

**算法描述：** 通过数学计算而非遍历，实现 O(1) 时间复杂度的日期匹配

**算法实现：**

```
def occurs_on_date(self, target_year, target_month, target_day):
    """
    判断事件是否在指定日期发生
    """
    # 检查排除列表
    if (target_year, target_month, target_day) in self.excluded_dates:
        return False

    # 检查结束日期限制
    if self.end_year is not None:
        target_date_num = target_year * 10000 + target_month * 100 +
target_day
        end_date_num = self.end_year * 10000 + self.end_month * 100
+self.end_day
        if target_date_num > end_date_num:
            return False

    # 计算天数差
    days_diff = date_diff_days(self.start_year, self.start_month,
self.start_day,
                       target_year, target_month, target_day)

    if days_diff < 0:
        return False

    # 根据重复类型判断
    if self.repeat_type == RepeatType.ONCE:
        return days_diff == 0

    elif self.repeat_type == RepeatType.DAILY:
        # 每日重复：任意天数差都可以
        if self.repeat_count and days_diff >= self.repeat_count:
            return False
        return True

    elif self.repeat_type == RepeatType.WEEKLY:
        # 每周重复：天数差必须是7的倍数
```

```
        if days_diff % 7 != 0:
            return False
        week_occurrence = days_diff // 7
        if self.repeat_count and week_occurrence >= self.repeat_count:
            return False
        return True

    elif self.repeat_type == RepeatType.MONTHLY:
        # 每月重复：计算月份差
        months_diff = (target_year - self.start_year) * 12 + (target_month
-self.start_month)
        if months_diff < 0:
            return False
        if self.repeat_count and months_diff >= self.repeat_count:
            return False

        # 处理边界：如开始日期是31日，但目标月份只有30天
        target_max_day = get_days_in_month(target_year, target_month)
        adjusted_day = min(self.start_day, target_max_day)
        return target_day == adjusted_day

    elif self.repeat_type == RepeatType.YEARLY:
        # 每年重复
        years_diff = target_year - self.start_year
        if years_diff < 0:
            return False
        if self.repeat_count and years_diff >= self.repeat_count:
            return False

        if target_month != self.start_month:
            return False

        # 特殊处理闰年2月29日
        if self.start_month == 2 and self.start_day == 29:
            max_day = get_days_in_month(target_year, 2)
            return target_day == max_day

        return target_day == self.start_day

    elif self.repeat_type == RepeatType.CUSTOM:
        # 自定义间隔：天数差必须是间隔的倍数
        if days_diff % self.custom_interval != 0:
            return False
        custom_occurrence = days_diff // self.custom_interval
        if self.repeat_count and custom_occurrence >= self.repeat_count:
            return False
        return True

    return False
```

**边界情况处理：**

1. **2月29日问题：** 非闰年用2月28日代替
2. **月末日期问题：** 如1月31日每月重复，2月用2月28/29日
3. **排除日期：** 支持从重复系列中排除特定日期

---

## 3.3 能查询农历日期

```
2.3.1 处已呈现
```

## 3.4 能按关键词搜索事件

```
2.4 处已呈现
```

---

## 3.5 支持给事件标🌟以示重要

**代码位置：** `models/event.py` 文件，`Event.starred` 属性
**实现方式：**

- 事件类中添加 `starred: bool` 属性
- UI显示：⭐ 前缀

---

# 4. 数据结构与持久化

## 4.1 事件类定义

**代码位置：** `models/event.py` #line16
实现思路：

```
class Event:
    """事件类"""
    def __init__(self, event_id, name, description,
start_year,start_month,start_day,repeat_type=RepeatType.ONCE,
custom_interval=1,repeat_count=None,starred=False, end_year=None,
end_month=None, end_day=None,excluded_dates=None):
        """
        初始化事件
        参数解释:
        event_id: 事件ID
        name: 事件名称
```

```
        description: 事件描述
        start_year: 开始年份
        start_month: 开始月份
        start_day: 开始日期
        repeat_type: 重复类型
        custom_interval: 自定义间隔天数
        repeat_count: 重复次数(None表示无限)
        starred: 是否标星
        end_year: 结束年份（用于限制重复范围）
        end_month: 结束月份
        end_day: 结束日期
        excluded_dates: 排除的日期列表
"""
```

### 4.2 持久化

**代码位置：** `storage/event_storage.py`
使用JSON文件存储
***EventStorage类**

```
class EventStorage:
    def __init__(self, data_file='data/events.json'):
        self.data_file = data_file
        self._ensure_data_directory()
```

几个函数：

- save_events(self, events) 保存事件列表到文件 #line24
- load_events(self) 从文件加载事件列表 #line37
- add_event(self, events, event) 添加事件并保存 #line52
- delete_event(self, events, event_id) 删除事件并保存 #line62
- update_event(self, events, event_id, updates) 更新事件并保存 #line72

---

## 5. 鲁棒性：

### 5.1 日期输入鲁棒性

```
位于`models/date_utils`文件
```

- parse_and_validate_date(year_str, month_str, day_str)和validate_date(year, month, day)共同验证日期输入的正确性

```python
def validate_date(year, month, day):
"""
验证日期是否合法
"""
if not (1 <= year <= 9999):
    return False, "年份必须在1-9999之间"
if not (1 <= month <= 12):
    return False, "月份必须在1-12之间"
max_day = get_days_in_month(year, month)
if not (1 <= day <= max_day):
    return False, f"该月的日期必须在1-{max_day}之间"
return True, ""
```

```python
def parse_and_validate_date(year_str, month_str, day_str):
"""
日期解析和验证函数
"""
# 检查是否为空
if not year_str or not month_str or not day_str:
    return False, None, None, None, "年份、月份、日期都不能为空"
# 去除空白字符
year_str = str(year_str).strip()
month_str = str(month_str).strip()
day_str = str(day_str).strip()
# 检查是否为空
if not year_str or not month_str or not day_str:
    return False, None, None, None, "年份、月份、日期都不能为空"
# 尝试转换为整数
try:
    year = int(year_str)
    month = int(month_str)
    day = int(day_str)
except ValueError:
    return False, None, None, None, "年份、月份、日期必须是整数"
# 日期验证
is_valid, error_msg = validate_date(year, month, day)
if not is_valid:
    return False, None, None, None, error_msg
return True, year, month, day, ""
```

---

## 5.2 事件重复次数及间隔输入鲁棒性

位于 `ui/event_manage_view.py` 文件

```python
# 自定义间隔
custom_interval = 1
if repeat_type == RepeatType.CUSTOM:
    try:
        custom_interval = int(self.custom_interval_var.get())
    if custom_interval < 1:
        messagebox.showerror("输入错误", "自定义间隔必须是正整数")
        return
    except ValueError:
        messagebox.showerror("输入错误", "自定义间隔必须是整数")
        return

# 重复次数
repeat_count = None
if not self.infinite_var.get():
    repeat_count_str = self.repeat_count_var.get().strip()
        if not repeat_count_str:
        messagebox.showerror("输入错误", "请输入重复次数，或勾选无限重复")
        return
    try:
        repeat_count = int(repeat_count_str)
        if repeat_count < 1:
            messagebox.showerror("输入错误", "重复次数必须是正整数")
            return
        if repeat_count > 10000:
            messagebox.showerror("输入错误", "重复次数不能超过10000次")
            return
    except ValueError:
        messagebox.showerror("输入错误", "重复次数必须是整数")
        return
```

## 6. 写在最后的废话

发现写到最后已经没有力气写点什么了，希望没有漏掉什么没有写吧

## 辛苦看到这里！

## *完结撒花*